SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA10 – SOFTWARE ENGINEERING

ASSIGNMENT 1 – CO1

Design, Simulate and Validate an Agile Delivery Strategy for the Omnichannel Order Fulfilment Platform

CO1: Apply agile methodologies and project management techniques such as scale and lean to effectively manage software projects using industry-standard tools like Jira.

Student Name	ATHIN SIVA.S	**Register Number**	192524237
Department / Section	AIDS	**Academic Year / Semester**	2026
Faculty Name		**Nature of Assignment**	**Individual**
Duration	**1 Week**	**Total Marks**	**50**
Date of Submission	5/11/2026	**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Problem Investigation and Delivery Outcomes	5		
2	Agile and Lean Operating Model	6		
3	Prioritized Product Backlog (Jira / Selected Tool)	8		
4	Capacity-Based Release Plan and Workflow	7		
5	Delivery Simulation and Design Validation	7		
6	Scaled Delivery Redesign and Validation (Day-4)	12		
7	Individual Technical Reflection	5		
	TOTAL	**50**		

OVERALL REMARKS

Student Signature	Faculty Signature	Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

Task	Evidence Expected	Status	Note
1	Investigation questions, stakeholder findings / simulated responses, problem statement, scope boundaries, feasibility, measurable success criteria		
2	Comparison of two approaches, justified selection, operating model with Lean practices, tool investigation & selection		

3	≥4 epics, ≥12 user stories with AC, priorities, estimates, dependencies, DoR, DoD, stakeholder conflict decisions, tool screenshots/export		
4	Capacity assumptions, 4 sprint goals, selected work, estimation method, dependencies (gateway/scanner), workflow + WIP limits, urgent/unfinished rules		
5	Bottleneck prediction, ≥10 simulated events (labelled), tool updates, ≥3 delivery indicators, interpretation + improvement for next sprint		
6	Impact diagnosis, redesign objectives, revised backlog/roles/sequencing, scaled Agile practices, ≥5 further events, comparative indicators, defence		
7	Significant decision, evidence-driven change, remaining vulnerability, ethical concern addressed, next investigation linked to own artefacts		

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All tasks (1–7) answered with required evidence	☐ Yes ☐ Partly ☐ No
Investigation questions + stakeholder / simulated findings present (Task 1)	☐ Yes ☐ Partly ☐ No
Two delivery approaches compared and operating model justified (Task 2)	☐ Yes ☐ Partly ☐ No
≥4 epics + ≥12 stories with AC, priorities, estimates, DoR/DoD (Task 3)	☐ Yes ☐ Partly ☐ No
Capacity plan, sprint goals, workflow with WIP limits shown (Task 4)	☐ Yes ☐ Partly ☐ No
Sprint-1 simulation: prediction, ≥10 events, indicators, improvement (Task 5)	☐ Yes ☐ Partly ☐ No
Day-4 redesign: impact, objectives, revised strategy, validation, defence (Task 6)	☐ Yes ☐ Partly ☐ No
Individual reflection covering decision, change, vulnerability, ethics, next step (Task 7)	☐ Yes ☐ Partly ☐ No
Tool screenshots / export (backlog, board, workflow) included and labelled	☐ Yes ☐ Partly ☐ No
All simulated data clearly labelled as simulated (not presented as real delivery)	☐ Yes ☐ Partly ☐ No
No unfilled template placeholders; work consistent throughout	☐ Yes ☐ Partly ☐ No
Viva readiness (able to defend prioritization, capacity, WIP, indicators)	☐ Yes ☐ No

Task 1: Investigate the Problem and Define Delivery Outcomes

1.1 Problem statement and product goal

Customers, pickers, support agents and finance reviewers cannot trust order, stock or refund status. Stock shown as available fails at picking, tracking shows no milestones, refunds are slow and approved informally, and nobody clearly owns an order between stages.

Product goal: make an order traceable from checkout to delivery and, where needed, to an authorized and audited refund, backed by stock that customers and pickers can trust.

1.2 Investigation questions

- Q1. Why does stock shown as available fail at picking?
- Q2. Which rule decides who may approve a refund, and up to what amount?
- Q3. What must happen to an order when the payment gateway is unavailable?
- Q4. Which order milestones must customers and support agents see?
- Q5. What can seven people and ninety minutes of weekly stakeholder time deliver in eight weeks?

1.3 Stakeholder findings (SIMULATED responses)

Access to real stakeholders is not assumed, so the responses below are simulated and identified as such (see Figure 1).

Stakeholder	Question	Simulated response	Planning decision
Customer	Q4	Tracking says Processing for days, so I call support.	Show permitted milestones: paid, picked, dispatched, delivered.
Warehouse picker	Q1	Pick lists include items already sold through another channel.	Reserve stock at checkout with an expiry; pick list shows only reserved and paid lines.
Support agent	Q4	Most calls ask where the order or refund is.	Prioritize order lookup and visible return and refund status.
Finance reviewer	Q2	Large refunds get approved in chat; reconciliation is manual.	Role- and amount-based approval with an audit record.
Store manager	Q1	Unclear whether the store or warehouse owns a click-and-collect order.	Record a fulfilment owner on every order.
Developer	Q3, Q5	Sandbox arrives in week 4 and scanner in week 5.	Build a payment stub and contract first; defer scanner work.

1.4 Scope boundaries

In the first release: authenticated checkout with stock reservation; idempotent payment request (stub, then sandbox); pick list; dispatch and status transitions; customer tracking page; return request; one refund approval workflow; support order lookup; rule-based low-stock alert (a deterministic threshold, not machine learning); barcode confirmation if the scanner arrives on time; reproducible build.

Excluded (roadmap only): sales analytics, trained demand forecasting, recommendations, loyalty, multi-currency pricing, real payment processing, and any real customer or card data. A menu entry for a module does not make that module complete.

1.5 Measurable delivery-success criteria

These are test targets, not claims of production achievement.

ID	Criterion	Acceptance condition
SC1	Core journey	One synthetic order reaches Delivered and one return reaches Refunded, with an audit record for every valid transition, shown in an end-to-end demo.
SC2	Stock integrity	100 concurrent test orders for a 10-unit item create at most 10 reservations; stock is never negative; repeated 5 times.
SC3	Payment idempotency	Ten retries with one payment key create one payment intent and one order.
SC4	Refund control	In at least 10 test cases every approval by a non-reviewer is rejected, and every approved refund logs approver, amount and time.
SC5	Reproducibility	A clean checkout builds and runs the journey from documented steps, and records survive a restart.

1.6 Feasibility

- **Technical:** synthetic products, orders and stock movements only. Database transactions and row locking can enforce SC2, and idempotency keys enforce SC3. The sandbox gateway is unavailable until week 4, so a local payment stub behind an agreed contract keeps work moving.
- **Operational:** warehouse and finance stakeholders give 90 minutes per week, so decisions (reservation expiry, refund limits, status names) are batched into refinement and review slots.
- **Financial:** no real charges. Existing development machines and free or student tool tiers are assumed; optional paid hosting would be estimated separately. The main cost is people time: 336 focused hours per sprint (Task 4).

Jira | FIQP - Order Fulfilment Platform

Forms / Stakeholder questionnaire - response summary (Task 1)

SIMULATED responses - illustrative mock-up. Replace with your own questionnaire or interview notes.

Stakeholder	Question	Simulated response	Planning decision
Customer	Q4 Milestones	Tracking says Processing for days, so I call support.	Show paid, picked, dispatched, delivered
Warehouse picker	Q1 Stock mismatch	Pick lists include items sold through another channel.	Reserve stock at checkout with expiry
Support agent	Q4 Milestones	Most calls ask where the order or refund is.	Order lookup, visible return and refund status
Finance reviewer	Q2 Refund rule	Large refunds approved in chat; reconciliation is manual.	Role- and amount-based approval with audit
Store manager	Q1 Ownership	Unclear who owns a click-and-collect order.	Record a fulfilment owner on every order
Developer	Q3, Q5 Constraints	Sandbox arrives week 4, scanner week 5.	Payment stub and contract first

Investigation questions: Q1 stock mismatch causes | Q2 refund approval rule | Q3 gateway unavailable | Q4 milestones to show | Q5 realistic eight-week scope

Success criteria derived: SC1 core journey, SC2 stock integrity, SC3 payment idempotency, SC4 refund control, SC5 reproducibility

Jira | FIQP - Order Fulfilment Platform

Forms / Stakeholder questionnaire - response summary (Task 1)

SIMULATED responses - illustrative mock-up. Replace with your own questionnaire or interview notes.

Stakeholder	Question	Simulated response	Planning decision
Customer	Q4 Milestones	Tracking says Processing for days, so I call support.	Show paid, picked, dispatched, delivered
Warehouse picker	Q1 Stock mismatch	Pick lists include items sold through another channel.	Reserve stock at checkout with expiry
Support agent	Q4 Milestones	Most calls ask where the order or refund is.	Order lookup, visible return and refund status
Finance reviewer	Q2 Refund rule	Large refunds approved in chat; reconciliation is manual.	Role- and amount-based approval with audit
Store manager	Q1 Ownership	Unclear who owns a click-and-collect order.	Record a fulfilment owner on every order
Developer	Q3, Q5 Constraints	Sandbox arrives week 4, scanner week 5.	Payment stub and contract first

Investigation questions: Q1 stock mismatch causes | Q2 refund approval rule | Q3 gateway unavailable | Q4 milestones to show | Q5 realistic eight-week scope

Success criteria derived: SC1 core journey, SC2 stock integrity, SC3 payment idempotency, SC4 refund control, SC5 reproducibility

Figure 1: Stakeholder questionnaire and findings summary (SIMULATED, illustrative mock-up)

Task 2: Design and Justify the Project Management Approach

2.1 Two delivery approaches

Dimension	Approach A: Scrum with Lean controls and XP practices	Approach B: Continuous Kanban with release milestones
Selecting work	Ordered backlog; sprint goal and scope negotiated at planning.	Pull the highest-value ready item whenever WIP allows.
Coordination	Daily 15-minute meeting, one board, short API design sessions.	Daily flow review of blocked and aging items.
Review	Sprint review and retrospective every two weeks.	Weekly service review and flow-metrics review.
Changing requirements	Changes negotiated at planning; urgent work swaps with equal planned work.	Re-prioritize any time; explicit expedite lane.
Cross-functional fit	A shared sprint goal keeps all seven roles on one outcome.	Good for flow, but no shared goal forcing roles to converge.
Limitations and risks	Overcommitment is likely with no velocity; ceremony overhead.	Weak time-boxes can push payment integration to the end; stakeholders get no fixed review point.

Selection: Approach A. The festival date is fixed, and a sprint boundary gives scarce warehouse and finance stakeholders a predictable review slot. The model borrows Kanban's WIP limits, pull and expedite policy, and XP's acceptance examples, pairing and automated tests, so queues in testing and integration become visible.

2.2 Operating model

Role	Responsibility
Product and analysis lead	Orders the backlog, resolves scope conflicts, runs stakeholder sessions, escalates blockers.
Frontend engineer	Tracking page, support lookup, accessibility checks.
Order API engineer	Order, status and return services; rotates as Scrum facilitator.
Inventory and data engineer	Stock reservation, pick list, low-stock rule, data model.
Payments and security engineer	Login, roles, idempotent payments, refund approval.
QA engineer	Acceptance tests; may reject a story that fails its criteria.
DevOps engineer	Build, deployment, environments, CI.

Cadence: sprint planning (2 h, day 1); daily meeting (15 min); weekly refinement inside the 90-minute stakeholder window (45 min with warehouse or finance, alternating); sprint review (60 min) and retrospective (45 min) on the last day; API or contract design huddle (30 min, on demand).

Escalation: a blocked item is flagged immediately. Unresolved after one working day, the product lead escalates; unresolved after two days or threatening the deadline, the lead negotiates scope with the sponsor. Engineers own completion evidence; the lead owns scope decisions.

2.3 Lean practices

Waste	Control	Evidence it works
Waiting	Payment stub and agreed contract before gateway access; one-day blocker escalation.	Blocked story-days

Waste	Control	Evidence it works
Handovers	One board, one status vocabulary; QA writes acceptance tests during refinement.	No duplicate status reports
Rework	Definition of Ready with a negative case; small stories; acceptance examples.	Reopened stories per completed story
Excess WIP	Limits of 4 in progress, 2 in review, 2 in test; finish before starting.	Maximum queue length; cycle time
Overproduction	Analytics deferred until order and stock data are trustworthy.	Deferred list stays deferred

2.4 Tool investigation and selection

Criterion	Jira	Trello	GitHub Projects
Planning	Epics, backlog, sprints, estimates, issue links.	Simple cards; sprints and estimates need add-ons.	Issues, iterations, custom fields.
Workflow	Custom statuses, transition rules, board WIP limits.	Lists; WIP limits via add-ons.	Status field; rules mostly via automation.
Collaboration	Comments, mentions, linked issues.	Very easy for small groups.	Native links to pull requests and commits.
Reporting	Burndown, velocity, control chart (check plan).	Limited.	Basic insights charts.
Accessibility	Web app; verify keyboard use; text labels, never colour alone.	Same checks.	Same checks.
Cost	Free tier exists; verify current limits.	Free tier exists; add-ons may cost.	Free for many uses; verify.

Selection: Jira. It supports epics, sprints, dependencies and transition rules in one tool, matching Scrum with WIP limits (the workflow is shown in Figure 2). **Limitations and responses:** (1) reporting may depend on the plan, so export CSV and calculate flow metrics in a spreadsheet (Appendix A); (2) the free-tier user cap may be below the 14 people in Task 6, so verify before scale-up and use a paid or educational plan if needed; (3) risk of mixing development states with order states, so Jira holds only the development workflow; (4) colour-coded labels need text equivalents for accessibility.

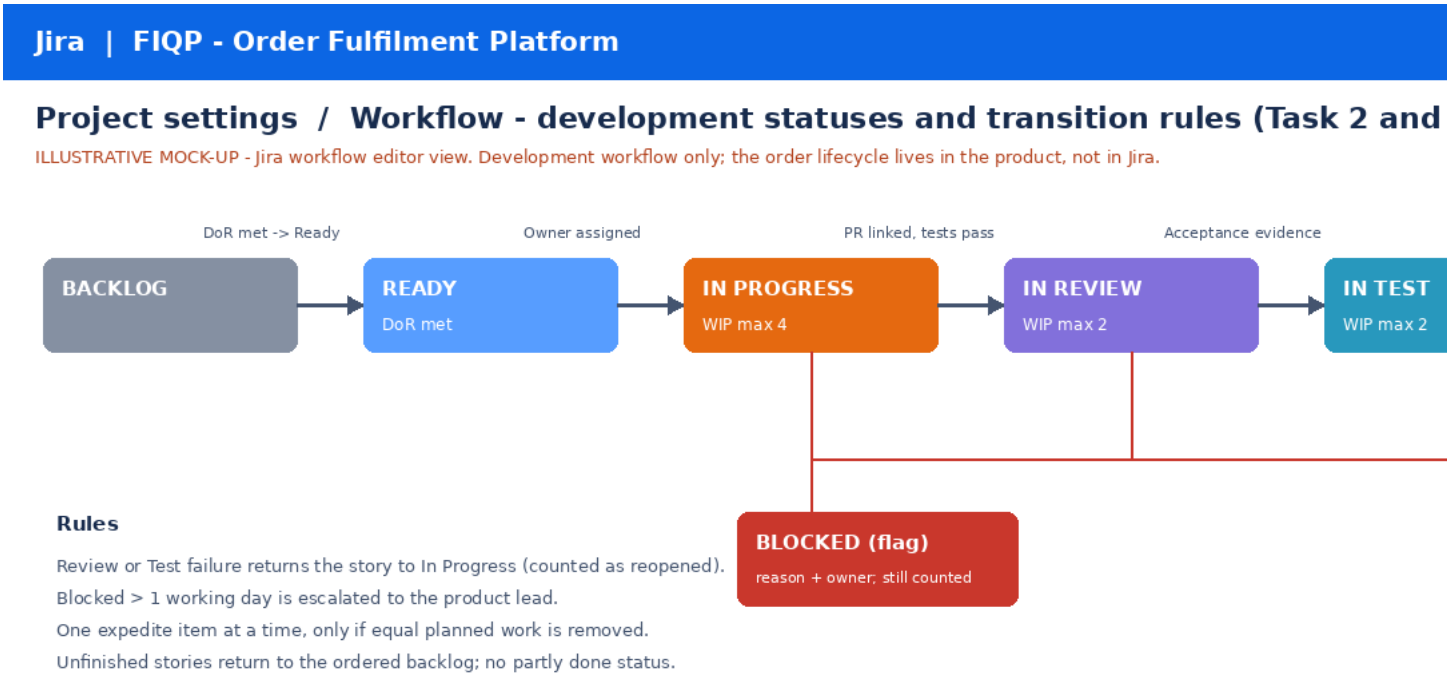


Figure 2: Jira workflow with statuses, transition rules and WIP limits (illustrative mock-up)

Task 3: Construct a Prioritized Product Backlog (Jira)

3.1 Prioritization method

MoSCoW, ordered within each class by (a) customer or financial harm if the story is missing, (b) the number of downstream stories depending on it, and (c) effort. Must means needed for the complete order-to-refund journey or prevents money or data loss. Estimates are relative story points (reference story S01 = 3; scale 1, 2, 3, 5, 8; stories above 5 are split), agreed by planning poker.

3.2 Modules retained or deferred

Module	Decision	Reason
Cart and checkout, catalogue	Retained (minimal)	Needed to place a synthetic order.
Inventory synchronization	Retained (reservation only)	Prevents overselling; pickers' top need.
Payment integration	Retained (stub, then sandbox)	An order cannot be confirmed without it.
Order tracking	Retained	Customer and support need.
Warehouse picking dashboard	Retained (pick list, then scan)	Connects stock to dispatch.
Returns and refunds	Retained (one approval path)	Finance control and the refund journey.
Support console	Retained (lookup only)	Reduces repeat calls.
Sales analytics	Deferred	Needs trustworthy order records first.

3.3 Epics and user stories

Epics: E1 Order and stock core; E2 Payments, returns and refund governance; E3 Warehouse fulfilment and tracking; E4 Reproducible delivery and support; E5 Deferred analytics. The Jira backlog is shown in Figure 3 and one open story in Figure 4.

ID	Epic	Story and acceptance criteria	Pri	Pts	Depends on	Owner
S01	E1	As a user I sign in and get only my role's permissions. AC: valid login gives role permissions; a customer calling a picker or refund function gets 403 on the server; session expires after an assumed 30 min idle.	M	3	none	Security
S02	E1	As a customer I place an order and stock is reserved. AC: reservation expires after an assumed 15 min; out-of-stock order rejected with a clear message; price calculated on the server.	M	5	S01	Order API
S03a	E2	As the system I send a payment request once. AC: repeated key gives one payment intent and one order; failure leaves the order recoverable as payment pending.	M	2	S02	Payments
S03b	E2	As the system I integrate the gateway sandbox. AC: sandbox success and failure map to order states; reconciliation record stored; no real card data.	M	3	S03a, gateway wk 4	Payments
S04	E3	As a picker I see my pick list. AC: only reserved and paid lines for assigned orders; each line shows location; sold-elsewhere items excluded.	M	3	S02	Inventory
S05	E3	As staff I dispatch an order. AC: only valid transitions (paid, picked, dispatched, delivered) succeed; each is audited with user and time; invalid transitions rejected.	M	5	S03a, S04	Order API
S06	E3	As a customer I see my order progress. AC: permitted milestones only, no internal notes, no other customer's orders; screen-reader labels; keyboard operable; status never by colour alone.	M	3	S05	Frontend
S07	E2	As a customer I request a return. AC: only the order owner; assumed 14-day window after Delivered; reason recorded.	M	3	S05	Order API

ID	Epic	Story and acceptance criteria	Pri	Pts	Depends on	Owner
S08	E2	As a finance reviewer I approve or reject a refund. AC: support agent cannot approve; reviewer decision and amount limit logged; above-limit refunds need a second approver; restock needs human confirmation.	M	5	S03b, S07	Security
S09a	E4	As a developer I build from a tagged commit. AC: clean checkout builds; health check passes.	M	3	S01, S02	DevOps
S09b	E4	As a team we run CI. AC: tests run on push; deployment notes current.	M	2	S09a	DevOps
S10	E4	As a support agent I find an order. AC: search by order ID; status, return and refund status visible; read-only; personal data masked.	S	3	S05	Frontend
S11	E1	As a manager I get a low-stock alert. AC: fixed threshold (assumed 5 units) raises one alert per item; rule-based, not machine learning.	S	3	S04	Inventory
S12	E3	As a picker I confirm items by barcode. AC: valid scan confirms; wrong item rejected and logged; manual fallback exists.	S	3	S04, scanner wk 5	Inventory
S13	E5	Sales analytics. AC: totals reconcile with order records.	C	3	S05	Frontend

First-release essentials are S01 to S09b (Must, 37 points). S10 to S12 are Should (9 points) and S13 is Could and deferred.

3.4 Definition of Ready

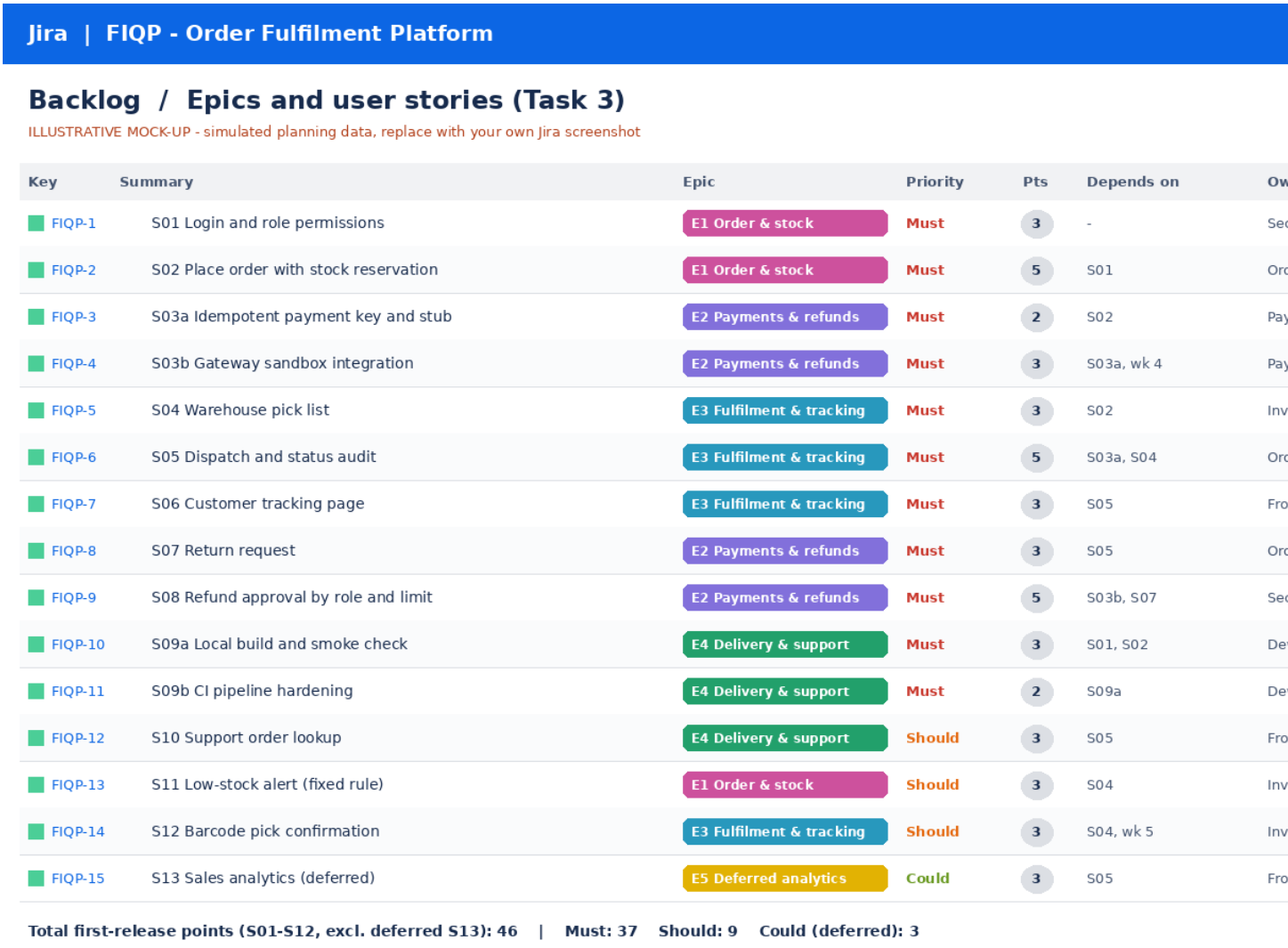
User and outcome understood; acceptance examples agreed, including one negative case; dependency identified and available or stubbed; privacy questions resolved (which personal data, who sees it); small enough for one sprint; estimated.

3.5 Definition of Done

Code reviewed; relevant automated tests pass; state persists and authorization is verified on the server; no real card or customer data in test evidence; user-facing screens have readable labels and keyboard access checked; deployment notes updated; evidence linked in Jira. UI completion alone is not Done.

3.6 Competing stakeholder requests

1. **Management (all nine modules) versus pickers (reservation first).** Reservation and pick list stay in Sprint 1 and analytics is deferred, because overselling creates failed orders and refunds that no dashboard can repair.
2. **Support agents (approve small refunds quickly) versus finance (auditable control).** Agents can view refund status but cannot approve; approval is a single reviewer click with a logged amount. A fast unauthorized refund is direct financial loss.
3. **Pickers (barcode scanning) versus scanner arrival (week 5).** S12 is Should and scheduled after the device arrives; manual confirmation covers the gap.



Jira | FIQP - Order Fulfilment Platform

Backlog / Epics and user stories (Task 3)

ILLUSTRATIVE MOCK-UP - simulated planning data, replace with your own Jira screenshot

Key	Summary	Epic	Priority	Pts	Depends on	Ow
FIQP-1	S01 Login and role permissions	E1 Order & stock	Must	3	-	Sec
FIQP-2	S02 Place order with stock reservation	E1 Order & stock	Must	5	S01	Ord
FIQP-3	S03a Idempotent payment key and stub	E2 Payments & refunds	Must	2	S02	Pay
FIQP-4	S03b Gateway sandbox integration	E2 Payments & refunds	Must	3	S03a, wk 4	Pay
FIQP-5	S04 Warehouse pick list	E3 Fulfilment & tracking	Must	3	S02	Inv
FIQP-6	S05 Dispatch and status audit	E3 Fulfilment & tracking	Must	5	S03a, S04	Ord
FIQP-7	S06 Customer tracking page	E3 Fulfilment & tracking	Must	3	S05	Fro
FIQP-8	S07 Return request	E2 Payments & refunds	Must	3	S05	Ord
FIQP-9	S08 Refund approval by role and limit	E2 Payments & refunds	Must	5	S03b, S07	Sec
FIQP-10	S09a Local build and smoke check	E4 Delivery & support	Must	3	S01, S02	Del
FIQP-11	S09b CI pipeline hardening	E4 Delivery & support	Must	2	S09a	Del
FIQP-12	S10 Support order lookup	E4 Delivery & support	Should	3	S05	Fro
FIQP-13	S11 Low-stock alert (fixed rule)	E1 Order & stock	Should	3	S04	Inv
FIQP-14	S12 Barcode pick confirmation	E3 Fulfilment & tracking	Should	3	S04, wk 5	Inv
FIQP-15	S13 Sales analytics (deferred)	E5 Deferred analytics	Could	3	S05	Fro

Total first-release points (S01-S12, excl. deferred S13): 46 | Must: 37 Should: 9 Could (deferred): 3

Figure 3: Jira backlog with epics, priorities, points and dependencies (illustrative mock-up)

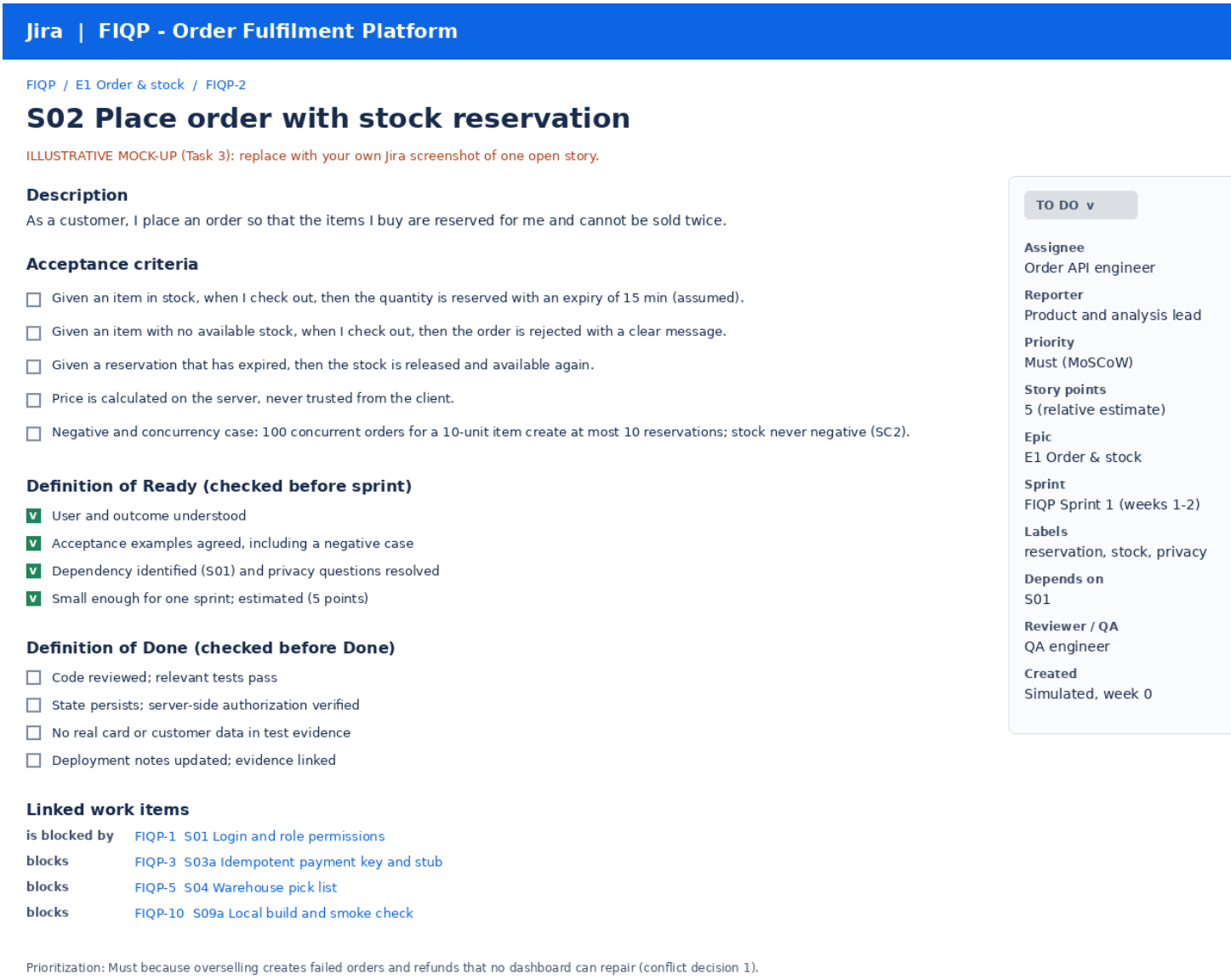


Figure 4: Story S02 open, with acceptance criteria, priority, estimate, dependency and owner (illustrative mock-up)

Task 4: Capacity-Based Release Plan and Workflow

4.1 Capacity assumptions

Seven people x 10 working days x 6 focused hours = 420 nominal hours per sprint. A 20% reserve for reviews, unplanned defects and integration leaves **336 focused hours**. Story points express relative effort and uncertainty and are never converted to hours. With no velocity, the Sprint 1 ceiling of **16 points** is a team judgement from comparing stories with the reference story (about five stories of average size 3), and it is replaced by observed results. One QA engineer is the scarcest resource.

4.2 Capacity negotiation

The first draft of Sprint 1 (S01, S02, S03, S04, S09 = 3+5+5+3+5 = 21 points) exceeded the ceiling. S03 is split into S03a (2, key and stub) and S03b (3, sandbox, waits for week 4); S09 is split into S09a (3) and S09b (2, moved to Sprint 2). The initial backlog is therefore not treated as a promise.

4.3 Sprint plan

Sprint	Goal	Stories	Points	Constraint
1 (weeks 1-2)	Secure ordering foundation	S01, S02, S03a, S04, S09a	16	Payment stub; no gateway needed.
2 (weeks 3-4)	Dispatch and stock alerts	S05, S06, S11, S09b	13	S06 starts when S05 reaches Test.
3 (weeks 5-6)	Sandbox, scanning, controlled refund	S03b, S07, S08, S12	14	Sandbox open since week 4; scanner arrives week 5.
4 (weeks 7-8)	Support visibility, hardening, rehearsal	S10 plus carryover	3	Deliberately light for regression and release rehearsal.

Dependency chain: S01 -> S02 -> S03a -> S05; S02 -> S04 -> S05, S11, S12; S05 -> S06, S07, S10; S03b and S07 -> S08. The week-4 gateway gates S03b, and the week-5 scanner gates S12. Total planned: 46 points; S13 is deferred.

4.4 Development workflow in Jira (separate from the order lifecycle)

The workflow and WIP limits are shown in Figures 2 and 5.

Status	Entry rule	Exit check
Backlog	Created.	Definition of Ready met.
Ready	DoR met, estimated.	Pulled when WIP allows.
In Progress	Owner assigned.	Code and unit tests complete.
In Review	Pull request linked.	Reviewer approves.
In Test	Acceptance evidence attached.	QA passes the acceptance criteria.
Blocked	Flag with reason and owner.	Blocker removed; returns to the prior status.
Done	All DoD items met.	None.

WIP limits: 4 in progress (seven people, with pairing and reviews taking time), 2 in review, 2 in test (one QA engineer makes this the tightest control). A blocked item stays counted.

Policies. Blocked: flag it, name an owner, escalate after one working day. Urgent: one expedite item at a time, allowed only if the lead removes equal planned work. Unfinished: returns to the ordered backlog and is re-estimated; never marked partly done and not counted as new velocity twice.

Operational order lifecycle (what the product implements, never a Jira status): Placed -> Reserved -> Paid -> Picked -> Dispatched -> Delivered -> Return requested -> Refund approved -> Refunded. Story S05 implements these transitions; the Jira statuses above track only the work.

4.5 Coherence with the product goal

S01 and S02 create a trusted order, S03 confirms payment safely, S04 and S05 move it through picking and dispatch, S06 and S10 make status visible, and S07 and S08 complete the return and audited refund. Remove any link and the journey in SC1 breaks, which is why analytics, not these links, is deferred.



Figure 5: Sprint 1 board at planning with WIP limits 4, 2, 2 and workflow rules (illustrative mock-up; simulated)

Task 5: Simulate Delivery and Validate the Design (SIMULATED)

5.1 Prediction before simulation

The likely bottleneck is the test queue and the order-to-payment hand-off, because reservation, persistence and payment keys converge there and only one QA engineer exists. **Control rule:** a WIP limit of 2 in Test and finish-before-start. **Evidence the rule works:** the Test queue never exceeds 2, and no story waits more than one day in Review or Test.

5.2 Sprint 1 event log (SIMULATED)

Figure 6 shows the board on day 7; Figure 7 shows it at the end of the sprint.

#	Day	Event	Jira update
1	1	Sprint planning; start S01 and S02	Two items In Progress.
2	2	Clarify reservation expiry with warehouse	S02 acceptance examples agreed (15 min).
3	3	S01 implementation complete	S01 to In Review.
4	4	Review finds missing expired-session test	S01 back to In Progress (reopen 1).
5	4	Start S04 on stock stub; start S09a	Four items In Progress (limit reached).
6	5	S01 passes review	S01 to In Test.
7	6	S02 code complete; QA passes S01	S02 to In Review; S01 Done (cycle 5 days).
8	7	Concurrency defect: two orders reserve the last unit	S02 back to In Progress (reopen 2).
9	7	S03a needs the fixed S02 order record	S03a flagged Blocked (starts counting).
10	8	S04 reviewed and tested	S04 Done (cycle 4 days).
11	9	Reservation lock repaired; S02 passes	S02 Done (cycle 8 days); S03a unblocked after 2 days.
12	10	S09a passes smoke check but deployment notes are missing	Fails DoD; stays open.
13	10	Sprint review	S03a and S09a carried over (5 points).

5.3 Indicators (SIMULATED)

The indicators were computed from a Jira-style CSV export with the script in Appendix A (Figure 8).

Indicator	Result	Interpretation
Completed vs planned	11 of 16 points = 68.8%	S01, S02, S04 done; S03a and S09a (5 points) carried over. Not proof of real productivity.
Blocked time	2 story-days (S03a)	Sequencing of the dependency was too loose.
Average cycle time	(5 + 8 + 4) / 3 = 5.7 days	S02 rework inflates it.
Max WIP / Test queue	4 in progress / 1 in test	Limits were respected; the queue did not become the bottleneck.
Reopened work	2 of 3 completed = 0.67	Ready checks lacked negative and concurrency cases.

5.4 Interpretation and improvement

The prediction was only partly right: the Test queue stayed short, and the real bottleneck was the S02 to S03a dependency. **Limitation of the original plan:** a downstream story was scheduled to start before its upstream story was stable, and the Definition of Ready did not demand a concurrency or negative case.

Improvements for Sprint 2: (1) a dependency-ready rule: a story may start only when its upstream story is in Test with a stable contract or stub; (2) negative and concurrency examples added to the Definition of Ready; (3) forecast from observed output (11 points) plus the 5-point carryover, with S05 split into S05a (3, status API) and S05b (2, dispatch action); (4) a daily API and QA pairing slot.

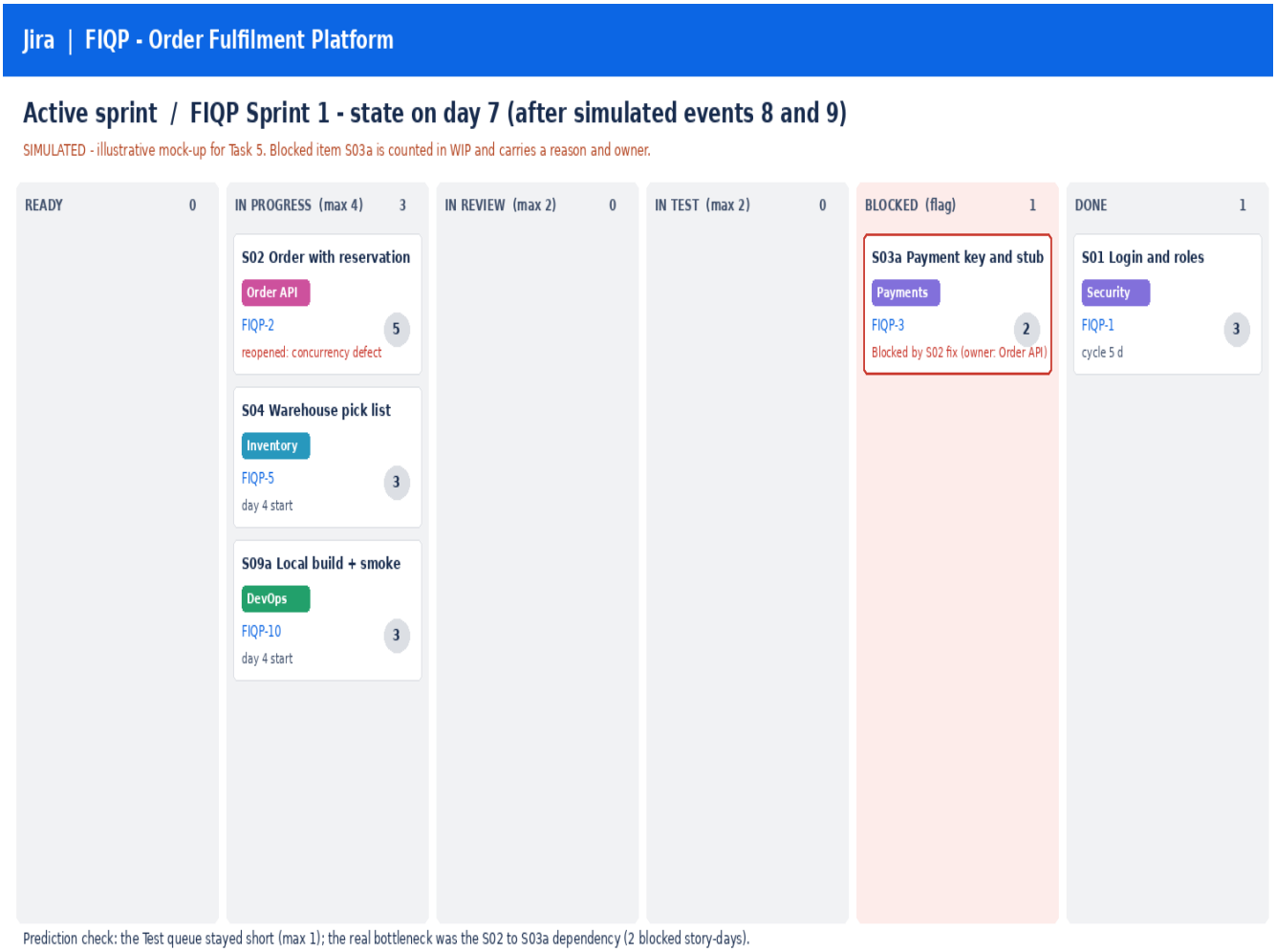
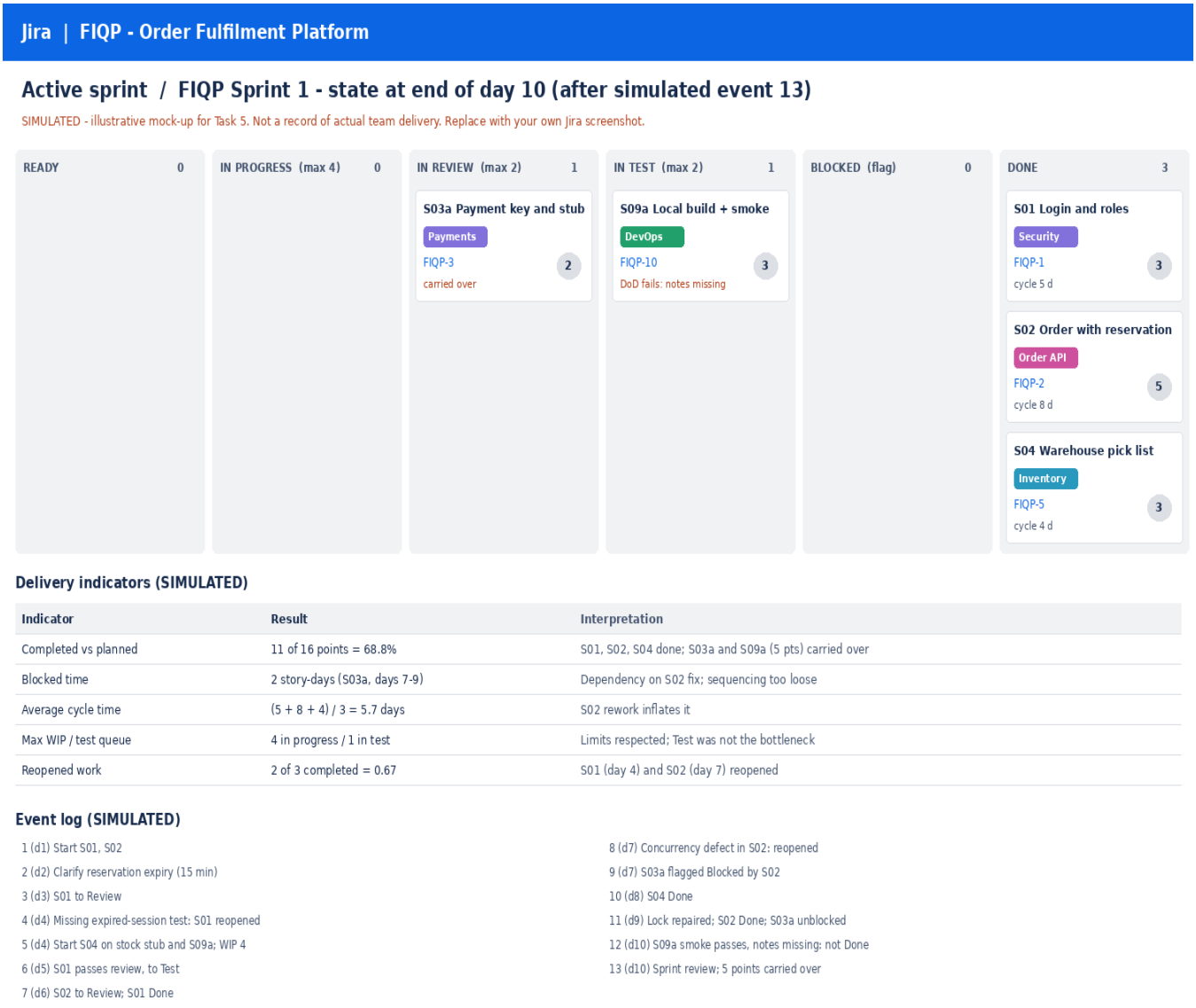


Figure 6: Sprint 1 board on day 7 with the Blocked item S03a (SIMULATED, illustrative mock-up)



Delivery indicators (SIMULATED)

Indicator	Result	Interpretation
Completed vs planned	11 of 16 points = 68.8%	S01, S02, S04 done; S03a and S09a (5 pts) carried over
Blocked time	2 story-days (S03a, days 7-9)	Dependency on S02 fix; sequencing too loose
Average cycle time	(5 + 8 + 4) / 3 = 5.7 days	S02 rework inflates it
Max WIP / test queue	4 in progress / 1 in test	Limits respected; Test was not the bottleneck
Reopened work	2 of 3 completed = 0.67	S01 (day 4) and S02 (day 7) reopened

Event log (SIMULATED)

1 (d1) Start S01, S02
2 (d2) Clarify reservation expiry (15 min)
3 (d3) S01 to Review
4 (d4) Missing expired-session test: S01 reopened
5 (d4) Start S04 on stock stub and S09a; WIP 4
6 (d5) S01 passes review, to Test
7 (d6) S02 to Review; S01 Done
8 (d7) Concurrency defect in S02: reopened
9 (d7) S03a flagged Blocked by S02
10 (d8) S04 Done
11 (d9) Lock repaired; S02 Done; S03a unblocked
12 (d10) S09a smoke passes, notes missing: not Done
13 (d10) Sprint review; 5 points carried over

Figure 7: Sprint 1 board at the end of the sprint with indicators and event log (SIMULATED, illustrative mock-up)

Excel | sprint1_export.csv (Jira export, SIMULATED data)

ILLUSTRATIVE MOCK-UP for Task 5 indicators - replace with your own Jira CSV export and spreadsheet.

Key	Story	Points	Status	Start day	Done day	Cycle (days)
FIQP-1	S01 Login and roles	3	Done	1	6	5
FIQP-2	S02 Order with reservation	5	Done	1	9	8
FIQP-3	S03a Payment key and stub	2	In Review	7		
FIQP-5	S04 Warehouse pick list	3	Done	4	8	4
FIQP-10	S09a Local build and smoke	3	In Test	4		

Indicators (computed by sprint_metrics.py, Appendix A)

Planned points	**16**
Completed points	**11**
Completion ratio	**68.8% (=11/16)**
Average cycle time (completed)	**5.7 days (=(5+8+4)/3)**
Reopened per completed story	**0.67 (=2/3)**
Blocked story-days	**2**

Figure 8: Spreadsheet export and computed indicators (SIMULATED, illustrative mock-up)

Task 6: Redesign Delivery Under Changed Conditions

A. Impact diagnosis

Assumption	Original	Change	Evidence and risk
Team	7 people	Team A has 6 (payments engineer leaves); Team B adds 7	Idempotency knowledge sits with one person (S03a had 2 blocked days).
Capacity	336 focused hours	More people, but onboarding and coordination	13 people is not twice the throughput.
Scope	One region	Two more warehouses; region tax, shipping and refund rules	New stories needed; refund approval (S08) is affected.
Dependencies	Gateway week 4, scanner week 5	Cross-team shared APIs	The S02 to S03a block shows the cost of tight coupling.
Deadline	Fixed	Unchanged	No schedule slack; scope must flex.

B. Redesign objectives (measurable)

1. One shared order and payment contract published by the end of Sprint 2.
2. Region rules held as configuration, with no duplicated order code.
3. One named integration owner; at most 2 cross-team dependencies in active implementation.
4. A demonstrable end-to-end journey in two regions at the Sprint 3 review (default rules), with region-specific rules by the Sprint 4 review.
5. Zero unresolved authorization defects at release; committed scope at most 90% of forecast capacity.

C. Revised strategy

Capacity. Team A: 6 x 10 x 6 = 360 nominal hours. Team B in Sprint 2 at 60% availability: 7 x 10 x 6 x 0.6 = 252. Total 612; a 30% reserve (original 20% plus 10% coordination) leaves about 428 hours against 336, roughly 1.3x. In Sprints 3 and 4, with Team B at 85% (357 hours), the total is 717 and about 502 hours remain, roughly 1.5x. For points, the simulated Sprint 1 output of 11 points from 7 people (about 1.6 per person), applied with the same availability and 10% extra coordination, gives roughly 14.7, 17.2 and 17.2 points for Sprints 2 to 4 (about 49 in total). This is a planning aid, not a promise.

New stories: S14 region tax and shipping configuration (5); S15 region refund-approval limits (3); S16 warehouse and region data model (3); S17 shared order and payment contract with contract tests (3); S18 payments knowledge transfer and replay tests (2). Total new: 16 points.

Deferrals: S13 stays deferred; S11 and S12 are deferred (picking stays manual); S10 becomes a stretch item. Committed scope is the Musts: 42 points against a forecast of about 49 (about a 14% buffer).

Team ownership: Team A takes order, stock, tracking and build; Team B takes region policy, payments, refund approval and UI. One Team B engineer pairs on the payment API in Sprint 2 before taking over the payments role, and Team B spends 40% of its Sprint 2 capacity on learning and integration.

Sprint	Goal	Team A	Team B	Points
2 (weeks 3-4)	Stabilize contract, dispatch status	S03a, S09a, S05a (8)	S17, S18 (5)	13
3 (weeks 5-6)	Two-region journey with sandbox and refund	S05b, S16 (5)	S03b, S07, S08 (11)	16
4 (weeks 7-8)	Region rules, tracking, hardening	S06, S09b (5)	S14, S15 (8)	13

Scaled Agile and Lean practices adopted (SAFe-inspired, without the full framework): a shared product backlog with one product lead; one 2-hour joint planning session at the start of Sprint 2; a dependency review twice weekly (30 minutes, like a scrum of scrums); one integrated demo per sprint; explicit interface ownership with one integration owner; contract-first development with consumer contract tests; a shared integration lane with WIP limit 2; and a coordinator who removes duplicate work before commitment. **Not adopted:** full Program Increment planning, release-train roles and extra ceremonies, since more ceremonies alone are not a redesign for two teams and eight weeks.

D. Validation: Sprint 2 under the new structure (SIMULATED)

#	Event	Revised behaviour
R1	Region requests a special tax and shipping rule	Recorded as S14, not pulled into the sprint; policy table design unchanged.
R2	Team B proposes a conflicting address payload	Contract review settles field names before coding; S17 published.
R3	Original payments engineer leaves (day 3)	Paired handover; replay tests preserve the idempotency contract; S03a done on day 3.
R4	Gateway sandbox access delayed again	Local adapter and replay tests continue; S03b flagged as a risk, no blocked effort.
R5	Integrated demo finds the status API exposes an internal notes field	S05a reopened; fixed but not yet retested, so carried over.
R6	S09a waits one day for a Team B environment	Counted as 1 blocked story-day; resolved at the dependency review.
R7	Midpoint capacity review	S11, S12 and S13 stay deferred; S10 remains a stretch item.

The revised plan and the comparison are shown in Figure 9.

Equivalent indicator	Original Sprint 1 (SIMULATED)	Revised Sprint 2 (SIMULATED)
Planned and completed points	16 planned; 11 completed	13 planned; 10 completed (S03a, S09a, S17, S18)
Completion ratio	68.8%	76.9%
Blocked story-days	2	1
Average cycle time	5.7 days	4.0 days
Reopened per completed story	2 / 3 = 0.67	1 / 4 = 0.25
Max cross-team active dependencies	not applicable	2

The sprints differ in scope, teams and carryover, so this comparison shows whether the designed mechanisms are plausible, not that real performance improved.

E. Defence of the decision

Benefits: clear ownership of contracts, region rules as configuration, earlier detection of defects at the integrated demo, and a preserved order-to-refund journey. **Limitations and trade-offs:** onboarding and coordination cost; barcode confirmation and low-stock alerts are deferred, so pickers confirm manually and managers check stock by report; region-specific rules arrive only in Sprint 4.

Feasibility under the same deadline: committed work is 42 points against an estimated 49, scope is cut by MoSCoW rather than by overtime, and a pre-agreed ladder applies if the midpoint review shows a shortfall: drop stretch S10; move S05b to Sprint 4; reduce S14 to flat per-region shipping. Never cut order integrity (S02),

idempotent payment (S03), status audit (S05), return (S07) or refund approval with audit and server-side authorization (S08).

Jira | FIQP - Order Fulfilment Platform (shared backlog, Team A + Team B)

Backlog / Revised release plan after the changed requirement (Task 6)

SIMULATED - illustrative mock-up. Not actual team delivery. Replace with your own Jira screenshot.

Sprint 2 (weeks 3-4)

Goal: Stabilize contract, dispatch status

Committed: 13 pts (forecast ~14.7)

TEAM A - order, stock, tracking, build

- S03a Payment key and stub (carryover) — 2
- S09a Local build + smoke (carryover) — 3
- S05a Status API — 3

TEAM B - region policy, payments, refund, UI

- S17 Shared order/payment contract — 3
- S18 Payments handover + replay tests — 2

Sprint 3 (weeks 5-6)

Goal: Two-region journey, sandbox, refund

Committed: 16 pts (forecast ~17.2)

TEAM A - order, stock, tracking, build

- S05b Dispatch action — 2
- S16 Warehouse/region data model — 3

TEAM B - region policy, payments, refund, UI

- S03b Gateway sandbox integration — 3
- S07 Return request — 3
- S08 Refund approval + audit — 5

Sprint 4 (weeks 7-8)

Goal: Region rules, tracking, hardening

Committed: 13 pts (forecast ~17.2)

TEAM A - order, stock, tracking, build

- S06 Customer tracking page
- S09b CI pipeline hardening

TEAM B - region policy, payments, refund, UI

- S14 Region tax and shipping config
- S15 Region refund limits

Stretch and deferred (unchanged deadline)

Stretch (not committed): S10 Support order lookup, 3 pts
Deferred: S11 Low-stock alert, S12 Barcode confirmation (3 + 3 pts)
Deferred: S13 Sales analytics (3 pts); promotions, forecasting (roadmap)
Committed Musts: 42 pts vs forecast ~49 pts (about 14% buffer)

Capacity (hours used for capacity only, not for points)

Sprint 2: A 6x10x6 = 360 h; B 7x10x6x60% = 252 h; total 612 h
30% reserve (20% + 10% coordination): ~428 h vs 336 h originally (~1.3x)
Sprints 3-4: B at 85% = 357 h; total 717 h; ~502 h (~1.5x), not 2x

Cross-team coordination (scaled Agile + Lean)

Shared product backlog and one product lead | joint planning at start of Sprint 2 | dependency review twice weekly (30 min)
One integration owner; integration lane WIP limit 2; max 2 cross-team dependencies active | one integrated demo per sprint
Contract-first (S17) with consumer contract tests; region rules stored as configuration (S14, S15, S16)

Comparison of equivalent indicators (SIMULATED)

Indicator	Original Sprint 1	Revised Sprint 2
Planned and completed points	16 planned; 11 completed	13 planned; 10 completed
Completion ratio	68.8%	76.9%
Blocked story-days	2	1
Average cycle time	5.7 days	4.0 days
Reopened per completed story	2 / 3 = 0.67	1 / 4 = 0.25
Max cross-team active dependencies	not applicable	2

Sprints differ in scope, teams and carryover: this shows plausibility of the mechanisms, not real improvement.

Figure 9: Revised two-team plan, capacity and indicator comparison (SIMULATED, illustrative mock-up)

Task 7: Individual Technical Reflection

Most significant decision. Protecting a complete order-to-refund journey (S01 to S09b) before adding module breadth, and deferring sales analytics (S13). This follows from the conflict decisions in Section 3.6 and keeps SC1 reachable.

Evidence changing the strategy. In the simulated event log (events 8, 9 and 11), a reservation defect in S02 blocked S03a for two story-days and reopened two of three completed stories. I therefore added a dependency-ready rule, negative and concurrency cases in the Definition of Ready, and, after the staffing change, a shared contract (S17) and an integration lane.

Remaining vulnerability. The sandbox gateway may behave differently from a live service, and my capacity figures come from one simulated sprint of 11 points. The Sprint 2 comparison covers different scope, so it cannot prove improvement.

Ethical concern. Privacy: S06 shows permitted milestones only, S10 masks personal data, and all evidence uses synthetic data. Accessibility: S06 requires screen-reader labels and no colour-only status. Workload transparency: carryover is shown on the board and not counted as new velocity twice. Simulations are labelled throughout.

Next enquiry before real use. Measure real order volume, stakeholder response time and gateway latency; validate the reservation expiry and refund limit with warehouse and finance staff; and check Jira plan limits for 14 users.

Sources and Tools Acknowledged

Scrum Guide (2020); Kanban Guide for Scrum Teams; Scaled Agile Framework public guidance; Atlassian Jira documentation; the course assignment brief. An AI assistant (Claude) helped draft this report and the illustrative figures. All figures are assumptions or simulations that the student must review, own and be able to explain. The Jira figures are illustrative mock-ups and should be replaced with screenshots from the student's own Jira project.

Appendix A: Supporting Code (illustrative)

The planning deliverable does not require code. These short listings support the stories and the indicators. A.1 and A.2 are illustrative sketches and have not been executed against a database; A.3 was run on the simulated export shown in Figure 8.

A.1 Atomic stock reservation and idempotent payment (SQL sketch for S02, S03a)

```
-- S02: reserve stock atomically (PostgreSQL sketch)
BEGIN;
SELECT available FROM stock WHERE sku = :sku FOR UPDATE;   -- row lock: two orders cannot take the last unit
UPDATE stock SET available = available - :qty
 WHERE sku = :sku AND available >= :qty;                   -- 0 rows updated => ROLLBACK, return "out of stock"
INSERT INTO reservation (order_id, sku, qty, expires_at)
VALUES (:order_id, :sku, :qty, now() + interval '15 minutes');
COMMIT;


-- S03a: idempotent payment request, same key never creates a second intent
INSERT INTO payment_intent (idempotency_key, order_id, amount, status)
VALUES (:key, :order_id, :amount, 'PENDING')
ON CONFLICT (idempotency_key) DO NOTHING;
SELECT * FROM payment_intent WHERE idempotency_key = :key; -- always returns the one existing intent
```

A.2 Refund approval rule (Python sketch for S08)

```
def can_approve_refund(role: str, amount: float, limit: float = 5000.0) -> bool:
    """Only a finance reviewer may approve, and only up to the limit.
    Above the limit a second approver is required (handled by the caller)."""
    return role == "finance_reviewer" and amount <= limit


def record_decision(audit_log, refund_id, approver, amount, decision):
    audit_log.append({"refund": refund_id, "approver": approver,
                      "amount": amount, "decision": decision})   # approver, amount, decision logged (SC4)
```

A.3 Sprint indicators from a Jira CSV export (Python, executed)

Input file sprint1_export.csv (SIMULATED data):

```
key,story,points,status,start_day,done_day,reopened,blocked_days
FIQP-1,S01 Login and roles,3,Done,1,6,1,0
FIQP-2,S02 Order with reservation,5,Done,1,9,1,0
FIQP-3,S03a Payment key and stub,2,In Review,7,,0,2
FIQP-5,S04 Warehouse pick list,3,Done,4,8,0,0
FIQP-10,S09a Local build and smoke,3,In Test,4,,0,0
```

Script sprint_metrics.py:

```
"""Compute Sprint 1 delivery indicators from a Jira-style CSV export (SIMULATED data)."""
```

```
import csv, sys


def load(path):
    with open(path, newline="") as f:
        return list(csv.DictReader(f))


def indicators(rows):
    planned = sum(int(r["points"]) for r in rows)
    done = [r for r in rows if r["status"] == "Done"]
    completed = sum(int(r["points"]) for r in done)
    cycle = [int(r["done_day"]) - int(r["start_day"]) for r in done]
    reopened = sum(1 for r in done if int(r["reopened"]) > 0)
    blocked = sum(int(r["blocked_days"]) for r in rows)
    return {
        "planned_points": planned,
        "completed_points": completed,
        "completion_pct": round(100 * completed / planned, 1),
        "avg_cycle_days": round(sum(cycle) / len(cycle), 1),
        "reopened_per_completed": round(reopened / len(done), 2),
        "blocked_story_days": blocked,
    }


if __name__ == "__main__":
    for k, v in indicators(load(sys.argv[1] if len(sys.argv) > 1 else "sprint1_export.csv")).items():
        print(f"{k}: {v}")
```

Output of: python sprint_metrics.py sprint1_export.csv

```
planned_points: 16
completed_points: 11
completion_pct: 68.8
avg_cycle_days: 5.7
reopened_per_completed: 0.67
blocked_story_days: 2
```